

SQL – DataWithBaraa

```
-- SELECT The Database
USE MyDatabase;

/*
=====
== SELECT Statement ==
=====
- SELECT is an SQL command used to fetch data from one or more tables in a database.
- It returns the result in the form of a result set (table).

-----
- syntax -
-----
SELECT column1, column2, ...
FROM table_name;
*/

-- select all records on the table
SELECT * FROM customers;

-- select specific columns from the table
SELECT id, first_name
FROM customers;

-- show available databases;
SHOW DATABASES;

-- determine the database
USE salesdb;

SELECT *
FROM orders;

-- select orderid, customerid
SELECT
    orderid,
    customerid
FROM orders;
```

```
/*
=====
== WHERE Clause ==
=====
- The WHERE clause in SQL is used to filter rows in a table based on a specific
condition.

-----
-- syntax --
-----
SELECT column1, column2
FROM table_name
WHERE condition;
*/

-- determine the database
USE MyDatabase;
```

```

-- select first 3 rows from the table
SELECT *
FROM customers
WHERE id <= 3;

-- select all customers from Germany
SELECT *
FROM customers
WHERE country = 'Germany';

-- select all customers that their score not equal 0
SELECT *
FROM customers
WHERE score != 0;

```

```

/*
=====
== ORDER BY Clause ==
=====
- It sorts the rows returned by a SQL query in ascending (ASC) or descending (DESC)
order based on one or more columns.

```

```

-----
-- syntax --
-----
    SELECT column1, column2
    FROM table_name
    ORDER BY column1 [ASC | DESC];
*/

```

```

USE MyDatabase;

```

```

-- sort the customers based on its 'id' from the biggest number to smallest number
SELECT *
FROM customers
ORDER BY id DESC;

-- retrieve all customers and sort the results by the heighest score first
SELECT *
FROM customers
ORDER BY score DESC;

-- retrieve all customers and sort the customers name from a-z
SELECT *
FROM customers
ORDER BY first_name ASC;

```

```

/*
=====
== GROUP BY Clause ==
=====
- The GROUP BY clause is used to group rows that have the same values in one or more
columns.
- used to combine rows with the same values, aggregate a column by another column
- COUNT(), MIN(), MMAX(), AVG(), SUM()

```

- note that: the none aggregated columns that written in the select must be written in the GROUP BY clause

```
-----
-- syntax --
-----
        SELECT column_name, aggregate_function(column_name)
        FROM table_name
        GROUP BY column_name;
*/

USE MyDatabase;

-- find total score for each country
SELECT
    country,
    SUM(score) AS total_score
FROM customers
GROUP BY country;

-- find number of customers for each country
SELECT
    country,
    COUNT(first_name) AS number_of_customers
FROM customers
GROUP BY country;

-- find the total score and total number of customers for each country
SELECT
    country,
    SUM(score) AS total_score,
    COUNT(first_name) AS total_customers
FROM customers
GROUP BY country;
```

```
/*
=====
== SELECT DISTINCT Statement ==
=====
- used to removes the duplicates

-----
-- syntax --
-----
        SELECT DISTINCT
            column1, column2, ..
        FROM table_name;
*/

USE MyDatabase;

-- removes duplicates names from the customers table
SELECT DISTINCT first_name
FROM customers;

-- return unique list of all countries
SELECT DISTINCT country
FROM customers;
```

```

/*
=====
== TOP & LIMIT ==
=====
- used to return the first n rows on the table

TOP: used in the SQL Server
*/

USE SalesDB;

-- get the first 2 orders
SELECT TOP 2 *
FROM Sales.Orders;

-- get the first 2 orders
SELECT TOP 2 *
FROM Sales.Employees;

-- get the top employee that get highest sales
SELECT TOP 1 *
FROM Sales.Employees
ORDER BY Salary DESC;

```

```

/*
=====
== Execution Order VS Coding Order ==
=====

-----
-- Coding Order --
-----
- syntax
    SELECT DISTINCT TOP <num>
    column1, column2, ...
    FROM table_name
    WHERE condition
    GROUP BY <column name>
    ORDER BY column_name ASC|DESC

-----
-- Execution Order --
-----
1- FROM Clause
2- WHERE Clause
3- GROUP BY Statement
4- HAVING Clause
5- SELECT DISTINCT Statement
6- ORDER BY Keyword
7- TOP Statement
*/

```

```

/*
=====
== CREATE Statement ==
=====
- used to create databases, tables

-----
- syntax -
-----
    CREATE TABLE table_name
    (column1 DATA_TYPE CONSTRAINTS, column2 DATA_TYPE CONSTRAINTS, ..);
*/

USE MyDatabase;

-- create a new table called persons with columns: id, person_name, birth_date, and
phone
CREATE TABLE persons (
    id INT NOT NULL,
    person_name VARCHAR(100),
    birth_date DATE,
    phone VARCHAR(50),
    CONSTRAINT pk_persons PRIMARY KEY (id)
);

```

```

/*
=====
== ALTER Statement ==
=====
- ALTER TABLE is used to modify an existing table in a database.
- used to add, delete, modify the columns and data types

-----
-- syntax --
-----
    ALTER TABLE table_name
    ALTER COLUMN column_name new_data_type [NULL | NOT NULL];
*/

USE NTI;

CREATE TABLE courses (id INT, name VARCHAR(100), price DECIMAL(10, 2));

SELECT * FROM courses;

-- add new column
ALTER TABLE courses
ADD description VARCHAR(255);

-- modify the column data type
ALTER TABLE courses
ALTER COLUMN price DECIMAL(12, 2);

-- drop a column
ALTER TABLE courses
DROP COLUMN description;

-- add primary key constraints
ALTER TABLE courses
ADD CONSTRAINT PK_courses_id PRIMARY KEY (id);

```

```

-- add unique constraints
ALTER TABLE courses
ADD CONSTRAINT UQ_courses_name UNIQUE (name);

INSERT INTO courses (id, name, price, description)
VALUES
(1, 'Python', 10.50, 'Learn Python programming'),
(2, 'C++', 25.00, 'Learn C++ programming'),
(3, 'C', 15.75, 'Learn C programming'),
(4, 'PHP', 30.20, 'Learn PHP programming'),
(5, 'MySQL', 8.99, 'Learn MySQL database');

```

```

/*
=====
== DROP Statement ==
=====
- used to delete the table from the database

DROP TABLE table_name;
*/

USE MyDatabase;

-- delete the table perons from the database
DROP TABLE persons;

```

```

/*
=====
== INSERT INTO Statement ==
=====
- used to append new data to the table

-----
-- syntax --
-----
INSERT INTO table_name (c1, c2, c3, ...) VALUES (v1, v2, v3, ...);
*/

SELECT * FROM users;

-- insert one record into the table
INSERT INTO users (id, name, address, email)
VALUES (1, 'osama', 'cairo', 'osama@gmail.com');

-- insert multiple records into the table
INSERT INTO users (id, name, address, email)
VALUES
(2, 'ahmed', 'cairo', 'ahmed@gmail.com'),
(3, 'khaled', 'giza', 'khaled@gmail.com'),
(4, 'ali', 'ayat', 'ali@gmail.com');

```

```
/*
=====
== INSERT using SELECT ==
=====
- must the output of the SELECT statement equal to the INSERT statement
- means number of columns must be equal in both tables
*/
```

```
/*
=====
== UPDATE Statement ==
=====
- used to update exist rows from the table

-----
-- syntax --
-----
    UPDATE table_name
    SET column_name new_value;
    WHERE condition;

-- note that: do not forget to write the WHERE condition
-- if not determined the WHERE condition will change all rows
*/
```

```
USE MyDatabase;
```

```
SELECT * FROM customers;
```

```
INSERT INTO customers VALUES (6, 'youssef', 'cairo', 450);
```

```
-- change the country of youssef to Egypt
UPDATE customers
SET country = 'Egypt'
WHERE first_name = 'youssef' AND country = 'cairo';
```

```
-- change the name of youssef to Ali
UPDATE customers
SET first_name = 'Ali'
WHERE first_name = 'youssef';
```

```
-- update all customers with a NULL score by setting their score = 0
UPDATE customers
SET score = 0
WHERE score IS NULL;
```

```

/*
=====
== DELETE Statement ==
=====
- used to delete specific rows from the table

-----
-- syntax --
-----
    DELETE FROM customers
    WHERE condition;

-- note that: do not forget the WHERE condition
-- if not determine the WHERE condition will delete all rows like TRUNCATE Statement
*/

```

```

USE MyDatabase;

```

```

SELECT * FROM customers;

```

```

-- remove all customers from Egypt

```

```

DELETE FROM customers
WHERE country = 'Egypt';

```

```

/*
-----
-- TRUNCATE --
-----
-- exactly like 'DELETE FROM table_name' without any conditions
-- used to delete all rows from the table

-----
-- syntax --
-----
    TRUNCATE TABLE table_name;
*/

```

```

TRUNCATE TABLE customers;

```

```

/*
=====
== HAVING Clause ==
=====
- used to filter aggregated data
- filter data after aggregation

-----
- syntax -
-----

```

```

*/

```

```

USE MyDatabase;

```

```

SELECT
    country,
    SUM(score) AS total_score
FROM customers
GROUP BY country
HAVING SUM(score) > 750;

```



```
/*
    find the average score for each country considering only customers with a score not
    equal 0
    and return only those countries with an average score greater than 430
*/
SELECT
    country,
    AVG(score) AS average_score
FROM customers
WHERE score != 0
GROUP BY country
HAVING AVG(score) > 430;
```

```
/*
=====
== Comparison Operators ==
=====
*/

USE MyDatabase;

SELECT * FROM customers;

-- find customer with id = 1
SELECT *
FROM customers
WHERE id = 1;

-- find customers with score > 700
SELECT *
FROM customers
WHERE score > 700;

-- find all customers with score < 700
SELECT *
FROM customers
WHERE score < 700;

-- find all customers that are not from USA
SELECT *
FROM customers
WHERE country <> 'USA';

-- find all customers that are not from USA
SELECT *
FROM customers
WHERE country != 'USA';
```

```

/*
=====
== Logical Operator ==
=====
- AND: all conditions must be true
- OR: one of the conditions must be true
- NOT: get the opposite of the condition
*/

Use MyDatabase;

-- find all customers that from USA and have an ID smaller than 3
SELECT *
FROM customers
WHERE country = 'USA' AND id < 3;

-- find all customers that from USA or have an ID smaller than 3
SELECT *
FROM customers
WHERE country = 'USA' OR id < 3;

-- find all customers that are not from USA
SELECT *
FROM customers
WHERE NOT country = 'USA';

```

```

/*
=====
== BETWEEN Operator ==
=====
- check if a value is within a range
*/

USE MyDatabase;

-- get the first 4 customers
SELECT *
FROM customers
WHERE id BETWEEN 1 AND 4;

-- find all customers that have score between 200 to 750
SELECT *
FROM customers
WHERE score BETWEEN 200 AND 750;

```

```

/*
=====
== Membership Operators ==
=====
*/

USE MyDatabase;

-- find all customers that from USA and UK
SELECT *
FROM customers
WHERE country IN ('USA', 'UK');

-- find all customers that have specific score 500, and 750

```

```
SELECT *
FROM customers
WHERE score IN (500, 750);
```

-- find all customers that are not from USA and UK

```
SELECT *
FROM customers
WHERE country NOT IN ('USA', 'UK');
```

```
/*
=====
== Like Operator ==
=====
- LIKE: used to search about sub-string, number, ... on the field
- There are two wildcards often used in conjunction with the LIKE operator:
    1- The percent sign % represents zero, one, or multiple characters
    2- The underscore sign _ represents one, single character
*/
```

```
USE MyDatabase;
```

-- search about customers from usa or USA

```
SELECT *
FROM customers
WHERE country LIKE 'USA';
```

-- search about counties that start with G

```
SELECT *
FROM customers
WHERE country LIKE 'G%';
```

-- Find customers whose names start with "A":

```
SELECT first_name, country, score
FROM customers
WHERE first_name LIKE 'A%';
```

-- Find customers whose names contain the letter "o"

```
SELECT first_name, country, score
FROM customers
WHERE first_name LIKE '%o%';
```

-- Find customers whose names end with "a"

```
SELECT first_name, country, score
FROM customers
WHERE first_name LIKE '%a';
```

-- Find customers whose names have exactly 4 characters

```
SELECT first_name, country, score
FROM customers
WHERE first_name LIKE '____'; -- Four underscores represent four characters
```

```

-- Find customers from countries whose name starts with "E"
SELECT first_name, country, score
FROM customers
WHERE country LIKE 'E%';

-- Find customers whose names have "a" as the second letter
SELECT first_name, country, score
FROM customers
WHERE first_name LIKE '_a%';

-- Find customers whose names have "a" as the second-to-last letter:
SELECT first_name, country, score
FROM customers
WHERE first_name LIKE '%a_';

-- Find customers whose names start with any letter between "A" and "D"
SELECT first_name, country, score
FROM customers
WHERE first_name LIKE '[A-D]%';

-- Find customers whose names contain either "a" or "e" and are from a country starting
with "E":
SELECT first_name, country, score
FROM customers
WHERE (first_name LIKE '%a%' OR first_name LIKE '%e%')
AND country LIKE 'E%';

-- Find customers whose names start with "A" or "B" and their score is greater than 750
SELECT first_name, country, score
FROM customers
WHERE (first_name LIKE 'A%' OR first_name LIKE 'B%')
AND score > 750;

-- Find customers whose names start with "S" or "N" and their country ends with "a" or
"e":
SELECT first_name, country, score
FROM customers
WHERE (first_name LIKE 'S%' OR first_name LIKE 'N%')
AND (country LIKE '%a' OR country LIKE '%e');

-- Find customers whose name contains "i" or "o" and their score is between 700 and 900:
SELECT first_name, country, score
FROM customers
WHERE (first_name LIKE '%i%' OR first_name LIKE '%o%')
AND score BETWEEN 700 AND 900;

-- Find customers from "Egypt" or "USA" whose names contain the letter "a" at any
position:
SELECT first_name, country, score
FROM customers
WHERE (country LIKE 'Egypt' OR country LIKE 'USA')
AND first_name LIKE '%a%';

```

```

-- Find customers whose names start with "E", "F", or "G" and score is greater than 750
SELECT first_name, country, score
FROM customers
WHERE (first_name LIKE 'E%' OR first_name LIKE 'F%' OR first_name LIKE 'G%')
AND score > 750;

```

```

-- Find customers whose names start with a vowel ("A", "E", "I", "O", or "U") and are
from the USA or Canada:
SELECT first_name, country, score
FROM customers
WHERE (first_name LIKE 'A%' OR first_name LIKE 'E%' OR first_name LIKE 'I%'
      OR first_name LIKE 'O%' OR first_name LIKE 'U%')
AND (country LIKE 'USA' OR country LIKE 'Canada');

```

```

-- Find customers whose names contain "a" or "o", their country contains "a", and their
score is between 700 and 900:
SELECT first_name, country, score
FROM customers
WHERE (first_name LIKE '%a%' OR first_name LIKE '%o%')
AND country LIKE '%a%'
AND score BETWEEN 700 AND 900;

```

```

-- Find customers whose names contain "o" in the second or second-to-last position:
SELECT first_name, country, score
FROM customers
WHERE first_name LIKE '_o%' OR first_name LIKE '%o_';

```

```

/*
=====
== INNER JOIN ==
=====
- we need a column that exists on the left table and right table
- return only matches rows from both tables

-----
-- syntax --
-----
    SELECT column1, column2, ...
    FROM table1
    INNER JOIN table2
    ON table1.common_column = table2.common_column;

```

```
*/
```

```

USE salesdb;
SHOW TABLES;

```

```

SELECT * FROM customers;
SELECT * FROM orders;

```

```

-- get all customers along with their orders, but only for customers who have placed
orde

```

```

SELECT
    c.firstname,
    c.country,
    o.orderid,

```

```

        o.orderdate,
        o.shipaddress
FROM customers AS c
INNER JOIN orders as o
ON o.orderid = c.customerid;

```

-- get all customers name, order id, order date and the sales of the orders, but only the customers who have placed on

```

SELECT
    c.firstname,
    o.orderid,
    o.orderdate,
    o.sales
FROM customers AS c
INNER JOIN orders AS o
ON o.orderid = c.customerid;

```

```

/*
=====
== LEFT JOIN ==
=====
- LEFT JOIN (also called LEFT OUTER JOIN) combines rows from two tables.
  It keeps every row from the left table.
- the left table is the primary source of your data
-----
-- syntax --
-----
    SELECT column1, column2, ...
    FROM table1
    LEFT JOIN table2
    ON table1.common_column = table2.common_column;
*/

```

```

-----
-- note that the orders of the table is very important --
-----

```

```

USE salesdb;

SELECT
    c.firstname,
    c.country,
    o.orderid,
    o.orderdate,
    o.sales
FROM customers AS c
LEFT JOIN orders AS o
ON o.orderid = c.customerid;

```

```

SELECT
    c.firstname,
    c.country,
    o.orderid,
    o.orderdate,
    o.sales
FROM orders AS o
LEFT JOIN customers AS c
ON o.orderid = c.customerid;

```

```

/*
=====
== RIGHT JOIN ==
=====
- RIGHT JOIN (or RIGHT OUTER JOIN) combines two tables such that:
  Every row from the right table appears.
  Only matching rows from the left table appear; unmatched ones show NULL.

-----
-- syntax --
-----
    SELECT column1, column2, ...
    FROM table1
    RIGHT JOIN table2
    ON table1.common_column = table2.common_column;

*/

USE salesdb;

SELECT * FROM customers;
SELECT * FROM orders;
SELECT
    c.firstname,
    c.score,
    o.orderid,
    o.orderdate
FROM customers AS c
RIGHT JOIN orders AS o
ON c.customerid = o.orderid;

```

```

USE MyDatabase;
/*
=====
== FULL JOIN ==
=====
- A FULL JOIN (or FULL OUTER JOIN) is a type of SQL join that returns all rows from
both tables, whether they have matching records or not.
-- If a row in Table A has a matching row in Table B, the result shows a single
combined row.
-- If a row in Table A does not have a match in Table B, the Table B columns are
filled with NULL.
-- If a row in Table B does not have a match in Table A, the Table A columns are
filled with NULL.

    What happens in a FULL JOIN:
    - LEFT JOIN part: All customers appear; if they have no orders, the order columns
are NULL.
      Carol has no order → orderid, orderdate, sales = NULL
    - RIGHT JOIN part: All orders appear; if the customer doesn't exist, the customer
columns are NULL.
      Order 103 has customerid = 4 → firstname, country = NULL
*/

USE MyDatabase;

/*
    What This Does
    - Returns all customers and all orders.

```

```
        - If a customer has no orders, OrderID, OrderDate, and sales will be NULL.
        - If an order has no matching customer (like customer_id = 6 in your orders
table),
        CustomerID, CustomerName, etc. will be NULL.
```

```
*/
```

```
SELECT
```

```
    c.id AS CustomerID,
    c.first_name AS CustomerName,
    c.country,
    c.score,
    o.order_id AS OrderID,
    o.order_date AS OrderDate,
    o.sales
```

```
FROM customers AS c
```

```
FULL OUTER JOIN orders AS o
```

```
ON c.id = o.customer_id;
```

```
/*
```

```
=====
```

```
== CROSS JOIN ==
```

```
=====
```

```
- combines every row from the left with every row from the right
- all possible combination
```

```
-----
```

```
-- syntax --
```

```
-----
```

```
SELECT *
```

```
FROM A CROSS JOIN B;
```

```
*/
```

```
USE MyDatabase;
```

```
-- generate all possible combinations of customers and orders
```

```
SELECT * FROM customers CROSS JOIN orders;
```

```
/*
```

```
=====
```

```
== 26 Multi-table JOIN ==
```

```
=====
```

```
- join more than two tables together
- LEFT JOIN keeps everything from the first (left) table.
- You can chain multiple LEFT JOINs.
```

```
-
```

```
- example:
```

```
    SELECT
```

```
        c.name,
        o.order_id,
        o.amount,
        p.method AS payment_method
```

```
FROM customers c
```

```
LEFT JOIN orders o
```

```
    ON c.customer_id = o.customer_id
```

```
LEFT JOIN payments p
```

```
    ON o.order_id = p.order_id;
```

```
*/
```



```

/*
    using SalesDB, retrieve a list of all orders, along with the related customer,
    product, and employee details. for each order display:
        order id, customer's name, product name, sales, price, sale's person
name
*/

```

-- to solve complex query like this, you should explore the 'ERD'

```

USE SalesDB;
SELECT * FROM Sales.Orders;
SELECT * FROM Sales.OrdersArchive;
SELECT * FROM Sales.Employees;
SELECT * FROM Sales.Products;

SELECT
    o.OrderID,
    o.Sales,
    c.FirstName AS CustomerFirstName,
    c.LastName AS CustomerLastName,
    p.Product,
    p.Price,
    e.FirstName AS EmployeeFirstName,
    e.LastName AS EmployeeLastName
FROM Sales.Orders AS o
LEFT JOIN Sales.Customers AS c
ON c.CustomerID = o.CustomerID
LEFT JOIN Sales.Products AS p
ON o.ProductID = p.ProductID
LEFT JOIN Sales.Employees AS e
ON e.EmployeeID = o.SalesPersonID;

```

```

/*
=====
== 27 UNION ==
=====
- used to combine the results of two or more SELECT queries into single result
set
- it removes duplicate rows by default
- all SELECT queries must have the same number of columns and compatible data
types
- ORDER BY is allowed only once at the end of the query
- data types of columns in each query must be compatible

-- union returns distinct rows from both queries
-- removes duplicate rows from the result

-----
-- syntax --
-----

    SELECT column1, column2, ...
    FROM table1
    WHERE condition1

    UNION

    SELECT column1, column2, ...
    FROM table2
    WHERE condition2;

*/

```

```
USE SalesDB;
```

```
-- the number of columns in the first table must be equal to the number of columns in  
the second table
```

```
-- the column names in the result set are determined by the column names specified in  
the first query
```

```
SELECT
```

```
    FirstName AS first_name,
```

```
    LastName AS last_name
```

```
FROM Sales.Customers
```

```
UNION
```

```
SELECT
```

```
    FirstName,
```

```
    LastName
```

```
FROM Sales.Employees;
```

```
-- combine the data from employees and customers into one table
```

```
SELECT
```

```
    CustomerID AS id,
```

```
    FirstName AS first_name,
```

```
    LastName AS last_name
```

```
FROM Sales.Customers
```

```
UNION
```

```
SELECT
```

```
    EmployeeID,
```

```
    FirstName,
```

```
    LastName
```

```
FROM Sales.Employees;
```

```
/*
```

```
-----  
-- UNION ALL --  
-----
```

```
- is used to combine results from multiple SELECT queries and keep all rows,  
including duplicates  
- it is faster than UNION because it does not remove duplicates
```

```
-----  
-- syntax --  
-----
```

```
        SELECT column1, column2
```

```
        FROM table1
```

```
        UNION ALL
```

```
        SELECT column1, column2
```

```
        FROM table2;
```

```
*/
```

```
-- combine the data from employees and customers into one table
```

```
SELECT
```

```
    CustomerID AS id,
```

```
    FirstName AS first_name,
```

```
    LastName AS last_name
```

```
FROM Sales.Customers
```

```
UNION ALL
```

```
SELECT
    EmployeeID,
    FirstName,
    LastName
FROM Sales.Employees;
```

```
/*
=====
== 28 EXCEPT ==
=====
- is a SET operator that returns rows from the first query that do NOT exist in
the second query
-----
-- syntax --
-----
        SELECT column1, column2
        FROM table1
        EXCEPT
        SELECT column1, column2
        FROM table2;
```

```
*/
```

```
USE SalesDB;
```

```
-- find the employees who are not customers at the same time
```

```
SELECT
    FirstName,
    LastName
FROM Sales.Customers
EXCEPT
SELECT
    FirstName,
    LastName
FROM Sales.Employees;
```

```
-- the order of queries in a EXCEPT does affect the result
```

```
SELECT
    FirstName,
    LastName
FROM Sales.Employees
EXCEPT
SELECT
    FirstName,
    LastName
FROM Sales.Customers;
```

```

/*
=====
== INTERSECT ==
=====
- is a SET operator that returns only rows that exists in BOTH queries
-----
-- syntax --
-----
        SELECT col1, col2 FROM table1
        INTERSECT
        SELECT col1, col2 FROM table2;
*/

```

```

USE SalesDB;

```

```

-- find the employees that are also customers

```

```

SELECT
    FirstName,
    LastName
FROM Sales.Employees
INTERSECT
SELECT
    FirstName,
    LastName
FROM Sales.Customers;

```

```

/*
| Operator | What it does |
|-----|-----|
| 'UNION' | Combines results and removes duplicates |
| 'UNION ALL' | Combines results and keeps duplicates |
| 'INTERSECT' | Returns only common rows in both queries |
| 'EXCEPT' | Returns rows from first query not in second |
*/

```

```

/*
=====
== String Functions ==
=====
- CONCAT(string1, string2, ...): combines multiple strings into one
- UPPER(text): convert all letters into upper case
- LOWER(text): convert all letters into lower case
- TRIM(text): remove the leading and trailing spaces from the left and right
side
- REPLACE(text, old, new): replaces specific character with a new character
- LEN(text): get the length of the characters
- LEFT(string, length): extracts specific number of characters from the start
- RIGHT(string, length): extracts specific number of characters from the end
- SUBSTRING(value, start, final_length): extracts a part of string at a
specified position
*/

```

```

USE MyDatabase;

```

```

-- concatenate first name and country into one column

```

```

SELECT
    first_name,
    country,
    CONCAT(first_name, '-', country) AS name_with_country

```

```

FROM customers;

-- you can write separator between columns
SELECT
    first_name,
    country,
    CONCAT(first_name, ' - ', country) AS name_with_country
FROM customers;

-- convert the first name to upper case
SELECT
    first_name,
    UPPER(first_name) AS upper_case
FROM customers;

-- convert the first name to lower case
SELECT
    first_name,
    LOWER(first_name)
FROM customers;

-- remove the spaces from the text
SELECT
    TRIM(' john ') AS trimmed_data
FROM customers;

-- remove the special characters from the text
SELECT
    TRIM('-' FROM '--osama--') AS trimmed_data
FROM customers;

-- replace word to another word
SELECT REPLACE('Hello World', 'World', 'SQL') AS Result;

-- replace the old phone syntax number to another syntax
SELECT
    '123-456-789' AS phone_number,
    REPLACE('123-456-789', '-', ' ') AS formatted_phone_number
FROM customers;

-- replace file extence from txt to csv
SELECT
    'file.txt' AS old_file,
    REPLACE('file.txt', '.txt', '.csv') AS new_file;

-- get the length of the text
SELECT
    first_name,
    LEN(first_name) AS text_length,
    LEN(TRIM(first_name)) AS text_after_trim
FROM customers;

-- extract the first 2 characters from the first name at the start
SELECT
    first_name,
    LEFT(first_name, 2) AS first_two_characters
FROM customers;

```

```

-- get the last character from the first name
SELECT
    first_name,
    RIGHT(TRIM(first_name), 1) AS last_character
FROM customers;

-- get the first two characters after trim
SELECT
    first_name,
    LEFT(TRIM(first_name), 2) AS first_two_characters
FROM customers;

-- get the first two characters before trim
SELECT
    first_name,
    LEFT(first_name, 2) AS first_two_characters
FROM customers;

-- retrieve a list of customers' first names after removing the first character
SELECT
    first_name,
    SUBSTRING(first_name, 2, 4) AS new_string
FROM customers;

-- retrieve a list of customers's first name after removing the first character
SELECT
    first_name,
    SUBSTRING(first_name, 2, LEN(first_name))
FROM customers;

-- make the first letter from the first_name capital, and the other letters small
SELECT
    UPPER(LEFT(first_name, 1)) + LOWER(SUBSTRING(first_name, 2, LEN(first_name))) AS
formatted_name
FROM customers;

SELECT
    first_name,
    LEN(first_name) AS 'length',
    LEN(TRIM(first_name)) AS 'length after trim',
    LEN(first_name) - LEN(TRIM(first_name)) AS 'flag'
FROM customers
WHERE LEN(first_name) != LEN(TRIM(first_name));

```

```

/*
=====
== 31 Number Functions ==
=====
- ROUND(): used to round a number to a specific number of decimal places
- ABS(): returns the absolute value of a number, removing any negative sign
- FLOOR() function rounds a number DOWN to the nearest integer.
- SQRT() function is used to calculate the square root of a number.
- CEILING() function rounds a number UP to the nearest integer.
- POWER() function is used to raise a number to a specific power (exponent).
- RAND(): Random number (0-1)
- PI(): PI() function returns the mathematical constant (pi).
*/

```

```

USE MyDatabase;

```

```

SELECT
    12.4567 AS 'number',
    ROUND(12.4567, 2) AS result;

--
SELECT
    12.6 AS 'number',
    ROUND(12.6, 0) AS result;

-- get the positive number
SELECT
    -120 AS 'number',
    ABS(-120) AS 'positive number';

-- get the positive number
SELECT
    -120.52 AS 'number',
    ABS(-120.52) AS 'positive number';

-- round the number up, and down
SELECT
    CEILING(9.1) AS up,
    FLOOR(9.9) AS down;

SELECT POWER(10, 2);

-- get the the square root of 10
SELECT SQRT(10) as 'square root';

-- get the nearest number
SELECT CEILING(12.1);

-- (2 × 2 × 2)
SELECT POWER(2, 3);

-- (5 × 5)
SELECT POWER(5, 2);

```

```

/*

```

```

=====
== 32 Date & Time Functions ==
=====

```

```

- Date Format: 2025-08-20
- Time Format: 18:55:45
- DateTime: 2025-08-20 18:55:45

```

```

- GETDATE(): returns the current system date and time of the SQL server

```

Format	Meaning
'ddd'	Wed
'MMM'	Jan

`yyyy`	2025	
`hh`	12-hour time	
`mm`	minutes	
`ss`	seconds	
`tt`	AM / PM	

```
*/
```

```
USE SalesDB;
```

```
SELECT
```

```
    OrderID,
    OrderDate,
    ShipDate,
    CreationTime
```

```
FROM Sales.Orders;
```

```
-- get current date and time
```

```
SELECT GETDATE() AS 'current DateTime';
```

```
/*
```

```
    -- insert the current date and time
    USE MyDatabase;
    INSERT INTO orders (order_date)
    VALUES (GETDATE());
```

```
*/
```

```
-- get the only date
```

```
SELECT CAST(GETDATE() AS DATE);
```

```
-- get the only time
```

```
SELECT CAST(GETDATE() AS TIME);
```

```
/*
```

```
=====
== Date Extraction ==
=====
DAY(): returns the day from a date
MONTH(): returns the month from a date
YEAR(): returns the year from a date
```

```
*/
```

```
USE SalesDB;
```

```
SELECT
```

```
    CreationTime,
    YEAR(CreationTime) AS 'Year',
    MONTH(CreationTime) AS 'Month',
    DAY(CreationTime) AS 'Day'
```

```
FROM Sales.Orders;
```

```
/*
```

```
DATEPART(part, date): returns a specific part of a date as a number
part: refers to the which part you need to extract like year, month, day, ...
date: refers to the date or the column that contains the date
```

```
*/
```

```
SELECT
```

```
    OrderDate,
    DATEPART(YEAR, OrderDate) AS 'Year',
```



```

        DATEPART(MONTH, OrderDate) AS 'Month',
        DATEPART(DAY, OrderDate) AS 'Day',
        DATEPART(WEEK, OrderDate) AS 'Week'
FROM Sales.Orders;

```

-- also can use the abbreviation of part

```

SELECT
    CreationTime,
    DATEPART(YY, CreationTime) AS 'Year',
    DATEPART(MM, CreationTime) AS 'Month',
    DATEPART(DD, CreationTime) AS 'Day',
    DATEPART(HOUR, CreationTime) AS 'Hour',
    DATEPART(MINUTE, CreationTime) AS 'Minute',
    DATEPART(SECOND, CreationTime) AS 'Seconds',
    DATEPART(MILLISECOND, CreationTime) AS 'Milli Seconds',
    DATEPART(MICROSECOND, CreationTime) AS 'Micro Seconds'
FROM Sales.Orders;

```

-- Get the Quarter

```

SELECT
    CreationTime,
    DATEPART(QUARTER, CreationTime) AS 'Quarter'
FROM Sales.Orders;

```

-- get the weekday

```

SELECT
    CreationTime,
    DATEPART(WEEKDAY, CreationTime) AS 'Weekday'
FROM Sales.Orders;

```

```

/*
    DATENAME(part, date): returns the name of a specific part of a date
    the data type of the output is string
*/

```

```

SELECT
    CreationTime,
    DATENAME(MONTH, CreationTime) AS 'Month_DateName',
    DATENAME(WEEKDAY, CreationTime) AS 'Weekday_DateName'
FROM Sales.Orders;

```

```

/*
    DATETRUNC(part, date): truncate the date to the specific part, will keep the
    part then reset the others values
    part: when define the part on DATETRUNC() function, anything after that will be
    reset
    if determine the part as MONTH, will reset the DAY as '01', because of there is
    no DAY like 00
    if determine the part as YEAR, will reset the MONTH as '01', because of there is
    no MONTH like 00
*/

```

```

SELECT
    CreationTime,
    DATETRUNC(YEAR, CreationTime) AS 'Hour'
FROM Sales.Orders;

```

```

/*
    count the number of orders based on the 'CreationTime'
    - will get one everywhere, because the level of details 'CreationTime' is
different
*/
SELECT
    CreationTime,
    COUNT(*) 'Number Of Orders'
FROM Sales.Orders
GROUP BY CreationTime;

-- aggregate the data based on the month level
SELECT
    DATETRUNC(MONTH, CreationTime) AS 'Month Level',
    COUNT(*) AS 'Number Of Orders'
FROM Sales.Orders
GROUP BY DATETRUNC(MONTH, CreationTime);

/*
    EOMONTH(date): return the last day of a month
    date: refers to the date or the column that contains the date
*/
SELECT
    CreationTime,
    EOMONTH(CreationTime) AS 'LastDay'
FROM Sales.Orders;

-- get the last day of the month
SELECT
    OrderDate,
    EOMONTH(OrderDate) AS 'LastDay'
FROM Sales.Orders;

-- get the start of month and end of month
SELECT
    CreationTime,
    DATETRUNC(MONTH, CreationTime) AS 'StartOfMonth',
    EOMONTH(CreationTime) AS 'EndOfMonth'
FROM Sales.Orders;

/*
    CAST(): used to convert the data types
*/
SELECT
    CreationTime,
    DATETRUNC(MONTH, CreationTime) AS 'StartOfDate',
    CAST(DATETRUNC(MONTH, CreationTime) AS DATE) AS 'StartOfData'
FROM Sales.Orders;

-- how many orders were placed each month
SELECT
    DATENAME(MONTH, OrderDate) AS OrderDate,
    COUNT(*) 'Number Of Orders'
FROM Sales.Orders
GROUP BY DATENAME(MONTH, OrderDate);

```

-- This query calculates how many orders were placed each month by grouping

```
SELECT
    YEAR(OrderDate) AS OrderYear,
    MONTH(OrderDate) AS OrderMonth,
    DATENAME(MONTH, OrderDate) AS MonthName,
    COUNT(*) AS NumberOfOrders
FROM Sales.Orders
GROUP BY
    YEAR(OrderDate),
    MONTH(OrderDate),
    DATENAME(MONTH, OrderDate)
ORDER BY
    OrderYear,
    OrderMonth;
```

-- show all orders that were placed during the month of february

```
SELECT *
FROM Sales.Orders
WHERE MONTH(OrderDate) = 2;
```

/*

```
=====
== Date Formatting & Casting ==
=====
- formatting: changing the format of a value from one to another (how the data
looks like)
  the formats can be as the following
      MONTH-DAY-YEAR
      DAY-MONTH-YEAR
-----
-- Syntax --
-----
FORMAT(value, format)
```

- casting: convert the data type from one to another (string -> integer)

*/

```
SELECT
    OrderDate,

    -- change the arrange of the Date
    FORMAT(OrderDate, 'dd/MM/yy') AS 'Order Date',

    -- get the short name of the day
    FORMAT(OrderDate, 'ddd') AS 'Day',

    -- get the full name of the day
    FORMAT(OrderDate, 'dddd') AS 'Day',

    -- get the month as number
    FORMAT(OrderDate, 'MM') AS 'Month',

    -- get the short name of the month
    FORMAT(OrderDate, 'MMM') AS 'Month',

    -- get the full name of the month
    FORMAT(OrderDate, 'MMMM') AS 'Month',

    -- get the year from the date as number
    FORMAT(OrderDate, 'yy') AS 'Year',
```

```

        -- get the short name of the year
        FORMAT(OrderDate, 'yyy') AS 'Year'
FROM Sales.Orders;

```

```

/*
    show the CreationTime using the following format:
    Day Wed Jan Q1 2025 12:34:56 PM
*/

```

```

SELECT
    CreationTime,
    CONCAT(
        'Day ',
        FORMAT(CreationTime, 'ddd MMM '),
        'Q', DATEPART(QUARTER, CreationTime), ' ',
        FORMAT(CreationTime, 'yyyy hh:mm:ss tt')
    ) AS [Creation Time]
FROM Sales.Orders;

```

```

USE SalesDB;

```

```

/*
    CONVERT(): converts a data or time value to a different data type & formats the
    value

```

```

    -----
    -- syntax --
    -----

```

```

    CONVERT(data_type, value, style[optional])
    style in CONVERT() controls how date/time or some numbers are formatted when
    converted to text
    style is a number code that tells SQL server which data formats to use when
    converting date/time to string

```

```

    -- data_type: usually VARCHAR or CHAR
    -- value: date or time
    -- style: format code

```

```

*/
SELECT
    CONVERT(VARCHAR, '123') AS [String to Int CONVERT],
    CONVERT(DATE, '2025-08-20') AS [DateTime to Date CONVERT],
    CreationTime,
    CONVERT(DATE, CreationTime) AS [DateTime To Date CONVERT],
    CONVERT(VARCHAR, CreationTime, 32) AS [USA Std. Style: 32],
    CONVERT(VARCHAR, CreationTime, 34) AS [EURO Std. Style: 34]
FROM Sales.Orders;

```

```

/*
    =====
    == 34 CAST() Function ==
    =====
    - CAST(): converts a value to a specified data type

```

```

    -----
    -- syntax --
    -----

```

```

    CAST(value AS data_type)
*/

```

```

USE SalesDB;

SELECT
    CAST('123' AS INT) AS [String To Integer],
    CAST(123 AS VARCHAR) AS [Integer To VARCHAR],
    CAST('2025-08-20' AS DATE) AS [String To Date],
    CAST('2025-08-20' AS DATETIME) AS [String To DateTime],
    CAST('2025-08-20' AS DATETIME2) AS [String To DateTime2],
    CAST(CreationTime AS DATE) AS [Datetime to DATE],
    CAST(CreationTime AS VARCHAR) AS [Datetime to Varchar]
FROM Sales.Orders;

```

```

/*
=====
== FORMAT ==
=====
*/

```

```

USE SalesDB;

-- format the date
SELECT
    FORMAT(GETDATE(), 'dd/MM/yyyy', 'en-GB') AS [Date];

```

```

-- format numbers -> add comma
SELECT
    FORMAT(1234567, 'N') AS [Number];

```

```

-- decimal places
SELECT
    FORMAT(1234.567, 'N2') AS [Number];

```

```

-- Simple date format
SELECT
    FORMAT(GETDATE(), 'yyyy-MM-dd') AS [Date];

```

```

-- custom format
SELECT
    FORMAT(GETDATE(), 'dd/MM/yyyy') AS [Date];

```

```

-- get the month name
SELECT
    FORMAT(GETDATE(), 'MMMM') AS [Month];

```

```

/*
=====
== 35 DATEADD ==
=====
- DATEADD(): is a SQL function used to add or subtract a time interval (days,
months, years, etc.) to/from a date.

```

```

-----
-- syntax --
-----

```

DATEADD(datepart, interval, date)

Parameters:

datepart: the unit of time (year, month, day, etc.)

number: how many units two add (use a negative number to subtract)

date: the original date

datepart	Meaning
year / yy	Year
month / mm	Month
day / dd	Day
hour / hh	Hour
minute / mi	Minute
second / ss	Second

*/

USE SalesDB;

-- add 10 days to 2026-01-29

SELECT

'2026-01-29',
DATEADD(day, 10, '2026-01-29');

-- subtract two months from 2026-01-29

SELECT

'2026-01-29',
DATEADD(month, -2, '2026-01-29');

-- add one year to 2026-01-29

SELECT

'2026-01-29',
DATEADD(year, 1, '2026-01-29');

-- Date after 30 days from today

SELECT

GETDATE(),
DATEADD(day, 30, GETDATE());

-- Calculates the delivery date 7 days after the order date.

USE MyDatabase;

SELECT

order_date,
DATEADD(day, 7, order_date) AS DeliveryDate
FROM Orders;

-- add one year to the CreationTime

USE SalesDB;

SELECT

CreationTime,
DATEADD(YEAR, 1, CreationTime) AS [Add One Year]
FROM Sales.Orders;

```

/*
=====
== 36 DATEDIFF ==
=====
- DATEDIFF() is a SQL function used to calculate the difference between two
dates.
    It returns the difference as an integer, based on the date part you specify
(days, months, years, etc.).

    allow us to find the differences between two dates

-----
-- syntax --
-----
- DATEDIFF(part, start_date, end_date)
*/

USE SalesDB;

SELECT
    DATEDIFF(month, '2025-01-15', '2026-01-15') AS MonthsDifference;

-- how many days
SELECT
    DATEDIFF(DAY, '2025-01-15', '2026-01-15') AS MonthsDifference;

-- calculate the age of employees
SELECT
    BirthDate,
    DATEDIFF(YEAR, BirthDate, GETDATE()) AS [Employee Age]
FROM Sales.Employees

/*
LAG() is a window (analytic) function in SQL that lets you look at a previous
row's
    value in the same result set without using a self-join.

    LAG(column_name, offset, default_value) OVER (PARTITION BY partition_column
ORDER BY order_column)
*/

/*
    find the average shipping duration in days for each month
    shipping duration = OrderDate - ShipDate
*/
SELECT
    MONTH(OrderDate) AS [Order Date],
    AVG(DATEDIFF(DAY, OrderDate, ShipDate)) AS [Day to Ship]
FROM Sales.Orders
GROUP BY MONTH(OrderDate);

/*
    find the number of days between each order and previous order
    this question called -> 'time gap analysis'
*/
SELECT
    OrderID,
    OrderDate AS [Current Order Date],

```

```

        LAG(OrderDate) OVER (ORDER BY OrderDate) AS [Previous Order Date],
        DATEDIFF(DAY, LAG(OrderDate) OVER (ORDER BY OrderDate), OrderDate) AS [Number of
Days]
FROM Sales.Orders;

```

```

/*
=====
== ISDATE ==
=====
- ISDATE() is a SQL Server function that checks whether a given expression can
be converted to a valid date.
It's useful for validating data before doing date operations.

-----
-- syntax --
-----
ISDATE(expression)

```

```
*/
```

```

SELECT
    ISDATE('2026-01-31'),
    ISDATE('31-01-2026'),
    ISDATE('hello');

```

```

/*
=====
== NULL Functions ==
=====
- NULL means no value / unknown / missing value.
- NULL is not equal to anything
- It does NOT mean: 0, empty string '', space ' ', It literally means "there is
nothing here".

```

```

-----
-- ISNULL() --
-----
- ISNULL() is a SQL Server function used to replace NULL with a value you
choose.

```

```

-----
-- COALESCE() --
-----
- COALESCE() returns the first NON-NULL value from a list of expressions.

syntax
COALESCE(value1, value2, value3, ...)

```

```
*/
```

```
USE SalesDB;
```

```

SELECT ISNULL(NULL, 0) AS [Result];      -- Result: 0
SELECT ISNULL(NULL, 'N/A') AS [Result];  -- Result: N/A
SELECT ISNULL(NULL, NULL) AS [Result];    -- Result: NULL

```

```
SELECT COALESCE(NULL, NULL, 10) AS [Result];  -- Result: 10
```



```
SELECT COALESCE(NULL, 'A', 'B') AS [Result];    -- Result: A
```

```
SELECT COALESCE(NULL, NULL, 0) AS [Result];      -- INT  
SELECT COALESCE(NULL, NULL, 'text') AS [Result]; -- VARCHAR
```

```
-- Show customer name and score. If score is NULL, show 0.
```

```
SELECT  
    first_name,  
    ISNULL(score, 0) AS score  
FROM customers;
```

```
-- Missing country, If country is NULL, show 'Unknown'.
```

```
SELECT  
    first_name,  
    ISNULL(country, 'Unknown') AS country  
FROM customers;
```

```
-- Fix messy names. Trim the name and replace NULL with 'No Name'.
```

```
SELECT  
    ISNULL(LTRIM(RTRIM(first_name)), 'No Name') AS first_name,  
    country,  
    score  
FROM customers;
```

```
-- Math with ISNULL(). Add 50 bonus points. If score is NULL treat it as 0.
```

```
SELECT  
    first_name,  
    score,  
    ISNULL(score, 0) + 50 AS score_with_bonus  
FROM customers;
```

```
-- Filtering with ISNULL. Show customers with score < 500. (Treat NULL score as 0).
```

```
SELECT  
    first_name,  
    country,  
    score  
FROM customers  
WHERE ISNULL(score, 0) < 500;
```

```
-- Grouping + ISNULL (important!). Show average score per country
```

```
SELECT  
    country,  
    AVG(ISNULL(score, 0)) AS avg_score  
FROM customers  
GROUP BY country;
```

```
-- Replace NULL score. If score is NULL, show 0.
```

```
SELECT  
    first_name,  
    COALESCE(score, 0) AS score  
FROM customers;
```

-- Multiple fallbacks (very realistic). If first_name is NULL -> use 'Unknown'.

```
SELECT
    COALESCE(first_name, 'Unknown') AS first_name,
    country,
    score
FROM customers;
```

-- Clean names + COALESCE. Some names have spaces (' John', ' hassan ').

```
SELECT
    COALESCE(NULLIF(LTRIM(RTRIM(first_name)), ''), 'No Name') AS clean_name,
    country,
    score
FROM customers;
```

-- COALESCE in math

```
SELECT
    first_name,
    score,
    COALESCE(score, 0) + 100 AS bonus_score
FROM customers;
```

-- Filtering with COALESCE. Show customers with score < 500 (NULL = 0).

```
SELECT
    first_name,
    country,
    score
FROM customers
WHERE COALESCE(score, 0) < 500;
```

-- Grouping + COALESCE. Average score per country (NULL = 0).

```
SELECT
    country,
    AVG(COALESCE(score, 0)) AS avg_score
FROM customers
GROUP BY country;
```

/*

```
    Why COALESCE is better here
    - Works in all databases
    - Handles multiple options
    - Safer data types than ISNULL
```

*/

-- find the average scores of the customers

```
USE MyDatabase;

SELECT
    id AS [customer id],
    score,
    COALESCE(score, 0) AS [score 2],
    AVG(score) OVER() AS [Average Score],
    AVG(COALESCE(score, 0)) OVER() AS [Average Score 2]
FROM customers;
```

```

-- handle the NULL before doing any mathematical operations
SELECT
    5 + NULL, -- Result = NULL
    5 + 5, -- Result = 10
    'A' + 'B', -- Result = AB
    'A' + NULL -- Result = NULL

/*
    display the full name of customers in a single field by merging their first and last
name
    and add 10 bonus points to each customer's score
*/
USE SalesDB;

SELECT
    CustomerID,
    FirstName,
    LastName,
    FirstName + ' ' + COALESCE(LastName, '') AS [Full Name],
    score AS [score],
    COALESCE(score, 0) + 10 AS [score with bonus]
FROM sales.Customers;

/*
-----
-- handle the NULL before doing any JOINS --
-----
if there are any NULL values inside any table, will loss the record at the output
therefore must handle the NULL before doing any JOINS
*/

```

```

/*
=====
== CASE Statement ==
=====
- It lets you add conditions inside a query and return different values based on
those conditions.
- evaluates a list of conditions and returns a value when the first condition is met
- the main purpose of CASE statement is "data transformation"
- drive new information [create new columns based on existing data]
- CATEGORIZING THE DATA -> group the data into different categories based on
certain conditions

-----
-- syntax --
-----
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN condition3 THEN result3
    ELSE default_result

*/

USE MyDatabase;

```

-- Categorize Customers by Score

```
SELECT
    first_name,
    score,
    CASE
        WHEN score >= 80 THEN 'High Score'
        WHEN score >= 50 THEN 'Medium Score'
        ELSE 'Low Score'
    END AS score_category
FROM customers;
```

-- Label Local vs Foreign Customers

```
SELECT
    first_name,
    country,
    CASE
        WHEN country = 'Egypt' THEN 'Local'
        ELSE 'Foreign'
    END AS customer_type
FROM customers;
```

-- Replace NULL Scores

```
SELECT
    first_name,
    CASE
        WHEN score IS NULL THEN 0
        ELSE score
    END AS cleaned_score
FROM customers;
```

-- Create Customer Rank Levels

```
SELECT
    first_name,
    score,
    CASE
        WHEN score >= 90 THEN 'VIP'
        WHEN score >= 70 THEN 'Gold'
        WHEN score >= 50 THEN 'Silver'
        ELSE 'Bronze'
    END AS customer_rank
FROM customers;
```

-- Conditional Aggregation - Count customers by score level:

```
SELECT
    COUNT(CASE WHEN score >= 80 THEN 1 END) AS high_score_customers,
    COUNT(CASE WHEN score BETWEEN 50 AND 79 THEN 1 END) AS medium_score_customers,
    COUNT(CASE WHEN score < 50 THEN 1 END) AS low_score_customers
FROM customers;
```

-- Create Pass/Fail Flag

```
SELECT
    first_name,
    score,
```

```

CASE
    WHEN score >= 50 THEN 'Passed'
    ELSE 'Failed'
END AS result
FROM customers;

```

-- Handle Multiple Countries with Labels

```

SELECT
    first_name,
    country,
    CASE
        WHEN country IN ('Egypt', 'Saudi Arabia', 'UAE') THEN 'Middle East'
        WHEN country IN ('USA', 'Canada') THEN 'North America'
        ELSE 'Other Region'
    END AS region
FROM customers;

```

```

USE SalesDB;

```

```

/*
    generate a report showing the total sales for each category
    - High: if the sales higher than 50
    - Medium: if the sales between 20 and 50
    - Low: if the sales equal or lower than 20
    and sort the result from lowest to highest
*/

```

```

SELECT
    Category,
    SUM(sales) AS [total sales]
FROM (
    SELECT
        OrderID,
        Sales,
        CASE
            WHEN Sales > 50 THEN 'High'
            WHEN Sales BETWEEN 20 AND 50 THEN 'Medium'
            WHEN Sales <= 20 THEN 'Low'
        END 'Category'
    FROM Sales.Orders
)t
GROUP BY Category
ORDER BY [total sales] ASC

```

-- Rules of CASE statement --

```

-- the data type of the results must be matching, means the results must be string,
numbers, dates, ...
-- CASE statement can be used anywhere in the

```

```

/*
=====
== Mapping Values ==

```

```

=====
- transform the values from one form to another in order to make it more readable
and re-usable for analytics
*/

```

```

-- retrieve employee details with gender displayed as full text

```

```

SELECT
    EmployeeID,
    FirstName,
    LastName,
    Gender,
CASE
    WHEN Gender = 'M' THEN 'Male'
    WHEN Gender = 'F' THEN 'Female'

    -- if there are NULL values
    ELSE 'Not Available'
END [Gender Info]
FROM Sales.Employees;

```

```

-- retrieve customer details with abbreviated country code

```

```

SELECT
    CustomerID,
    FirstName,
    LastName,
    Country,
CASE
    WHEN Country = 'Germany' THEN 'DE'
    WHEN Country = 'USA' THEN 'US'
    ELSE 'n/a'
END 'Country Abbreviation'
FROM Sales.Customers;

```

```

/*
=====
== Handle NULLs ==
=====
- replace NULLs with a specific values
- NULLs can lead to inaccurate results
- which can lead to wrong decision-making
*/

```

```

-- find the average scores of customers and treat NULLs as 0

```

```

SELECT
    CustomerID,
    FirstName,
    LastName,
    Score,
CASE
    WHEN Score IS NULL THEN 0
    ELSE Score
END [Clean Score],
AVG(Score) OVER () AS [AverageScore],
AVG(CASE
    WHEN Score IS NULL THEN 0
    ELSE Score

```

```

    END) OVER() AvgCustomerClean
FROM Sales.Customers;

/*
=====
== Conditional Aggregation ==
=====
- apply aggregate functions only on subsets of data that fulfill certain conditions
*/

```

```

-- count how many times each customer has made an order with sales greater than 30
SELECT
    CustomerID,
    SUM(CASE
        WHEN Sales > 30 THEN 1
        ELSE 0
    END) AS TotalOrdersHighSales,
    COUNT(*) AS TotalOrders
FROM Sales.Orders
GROUP BY CustomerID
ORDER BY CustomerID;

```

```

/*
=====
== Window Functions ==
=====
- perform calculations like aggregation on a specific subset of data without
losing the level of details of rows
- Returns all original rows and adds extra columns with aggregated/calculated
data
- we can use aggregate functions, Rank functions, value (analytics) functions

=====
== GROUP BY ==
=====
- Aggregate rows into groups based on one or more columns
- returns one row per group
- we can use aggregate functions

-----
-- window functions syntax --
-----

SELECT
    column1,
    column2,
    window_function_name(expression)
OVER (
    [PARTITION BY column]
    [ORDER BY column]
    [ROWS/RANGE frame]
) AS new_column
FROM table_name;

-----
-- window_function_name() --
-----

```

This is the function you want to use, like:

```
ROW_NUMBER()
RANK()
DENSE_RANK()
SUM()
AVG()
COUNT()
LAG()
LEAD()
```

-- Frame Clause (optional advanced part) --

- Controls which rows are included in the calculation.

-- most common syntax --

```
SELECT
    column,
    SUM(column) OVER (PARTITION BY column ORDER BY column) AS running_total
FROM table;
```

-- Simplest Example Syntax --

```
SELECT
    name,
    score,
    ROW_NUMBER() OVER (ORDER BY score DESC) AS row_num
FROM customers;
```

*/

USE SalesDB;

-- find total sales across all orders

```
SELECT
SUM(Sales) AS total_sales
FROM Sales.Orders;
```

-- find total sales for each product

```
SELECT
    ProductID,
    SUM(Sales) AS total_sales
FROM Sales.Orders
GROUP BY ProductID;
```

/*

- OVER(): tells SQL that the function used is a window function. it defines a window or subset of data

- PARTITION BY: divide the result set into windows (partitions)

*/


```
-- find the 'total sales across all orders', additionally provide details such order id, order date
```

```
SELECT
    OrderID,
    OrderDate,
    SUM(Sales) OVER() AS [total sales]
FROM Sales.Orders;
```

```
-- find the 'total sales for each product', additionally provide details such order id & order date
```

```
SELECT
    ProductID,
    OrderID,
    OrderDate,
    SUM(Sales) OVER(PARTITION BY ProductID) AS TotalSales
FROM Sales.Orders
ORDER BY OrderID ASC;
```

```
-- find the total sales across all orders
```

```
-- find the total sales for each product
```

```
-- additionally provide details such as order id & order date
```

```
SELECT
    ProductID,
    OrderID,
    OrderDate,
    Sales,
    -- find the total sales across all orders
    SUM(Sales) OVER() AS [Total Sales],
    -- find the total sales for each product
    SUM(Sales) OVER(PARTITION BY ProductID) AS [Total Sales By Products]
FROM Sales.Orders;
```

```
-- find the total sales for each combination of product and order status
```

```
SELECT
    ProductID,
    OrderID,
    OrderStatus,
    Sales,
    SUM(Sales) OVER(PARTITION BY ProductID, OrderStatus) [Sales by Products and Status]
FROM Sales.Orders;
```

```
-----

-- rank each order based on their sales from the highest to lowest
```

```
-- additionally provide details such order id & order date
```

```
SELECT
    OrderID,
    OrderDate,
    Sales,
    RANK() OVER(ORDER BY Sales DESC) AS [RankSales]
FROM Sales.Orders;
```

```
-----

-- Window Frame --
-----
```

```

-- define subset of rows with each window that is relevant for the calculation
-- Define which rows inside the partition are included in the calculation.

/*
    find running total sales across all orders, additionally provide order details
*/
SELECT
    OrderID,
    OrderDate,
    Sales,
    SUM(Sales) OVER(
        ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS RunningTotalSales
FROM Sales.Orders;

```

```

/*
    =====
    == SubQueries ==
    =====
    - is a query inside the main query

    - categories
    -----
    1) non-correlated subquery
    2) correlated subquery

    - result types
    -----
    1) scalar subquery: it returns only one single value
    2) row subquery: it can return multiple rows
    3) table subquery: return multiple rows and multiple columns

    where the subquery can be used within the main query
    -----
    1) SELECT:
    2) FROM:
    3) JOIN:
    4) WHERE: with [comparison operators, logical operators]

    -- ANY: Compares a value with any value returned by a subquery; true if at least
    one comparison is true.
        WHERE salary > ANY (SELECT salary FROM employees WHERE department = 'IT');

    -- ALL: Compares a value with every value returned by a subquery; true only if
    every comparison is true.
        WHERE salary > ALL (SELECT salary FROM employees WHERE department = 'IT');
*/

USE SalesDB;

-- scalar query example [single value, one row, and one column]
SELECT
    AVG(Sales) AS AverageSales
FROM Sales.Orders;

```

```

-- row subquery: return multiple rows and single column
SELECT
    CustomerID
FROM Sales.Orders;

-- table subquery: return multiple rows and multiple columns
SELECT
    CustomerID,
    OrderID,
    Sales
FROM Sales.Orders;

-----
-- subquery in 'FROM' clause --
-----

-- find the products that have a price higher than the average price of all products
-- main query
SELECT *
FROM (
    -- subquery
    SELECT
        ProductID,
        Price,
        AVG(Price) OVER() AveragePrice
    FROM Sales.Products
) AS t
WHERE Price > AveragePrice
ORDER BY Price;

-- rank the customers on their total amount of sales
SELECT
    *,
    RANK() OVER(ORDER BY TotalSales DESC) ASCustomerRank
FROM (
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
) t

/*
-----
-- subquery in 'SELECT' clause --
-----
- used to aggregate data side by side with the main query's data, allowing for
direct comparison
- must return a single value for each row of the main query
- commonly used to show aggregate/calculated values alongside main-query columns
- syntax
    SELECT
        column_name,

```

```

                                (SELECT column_name FROM table2 WHERE condition) AS
subquery_alias
                                FROM table1;
*/

-- show the product IDs, product names, prices, and the total number of orders
SELECT
    ProductID,
    Product,
    Price,
    ( SELECT COUNT(*) FROM Sales.Orders ) AS TotalOrders
FROM Sales.Products;

/*
-----
-- subquery in 'JOIN' clause --
-----
- used to prepare the data (filtering and aggregation) before joining it with
other tables
*/

-- show all customer details and find the total orders of each customer
SELECT
    *
FROM Sales.Customers AS c
LEFT JOIN (
    SELECT
        CustomerID,
        COUNT(OrderID) AS TotalOrders
    FROM Sales.Orders
    GROUP BY
        CustomerID
) o
ON c.CustomerID = o.CustomerID;

/*
-----
-- subquery in 'WHERE' clause --
-----
- used for complex filtering logic and makes query more flexible and dynamic
- only scalar subqueries are allowed to be used
- syntax
    SELECT
        Column1,
        Column2
    FROM table
    WHERE column = ( SELECT column FROM table2 WHERE condition )
*/

-- find the products that have a price higher than the average price of all products
SELECT *
FROM Sales.Products
WHERE Price > (
    SELECT
        AVG(Price) AS AveragePrice
    FROM Sales.Products
)

```

```
-- show the details of orders made by customers in Germany
SELECT
```

```
    *
FROM Sales.Orders
WHERE CustomerID IN (
    SELECT CustomerID
    FROM Sales.Customers
    WHERE Country = 'Germany'
)
```

```
-- show the details of orders made by customers in Germany
SELECT
```

```
    *
FROM Sales.Orders
WHERE CustomerID NOT IN (
    SELECT CustomerID
    FROM Sales.Customers
    WHERE Country != 'Germany'
)
```

```
-- find female employees whose salaries are greater than the salaries of any male
employees
```

```
SELECT
    EmployeeID,
    FirstName,
    Gender,
    Salary
FROM Sales.Employees
WHERE
    Gender = 'F'
    AND
    Salary > ANY ( SELECT Salary FROM Sales.Employees WHERE Gender = 'M' );
```

```
-- show all customer details and find the total orders of each customer
```

```
SELECT
    *,
    ( SELECT COUNT(*) FROM Sales.Orders AS o WHERE o.CustomerID = c.CustomerID ) AS
TotalOrders
FROM Sales.Customers AS c;
```

```
/*
```

```
-----
-- Non-correlated subquery
-- Runs independently from the main query.
-----
```

```
*/
```

```
-- Customers with a score above the average customer score.
```

```
SELECT *
FROM Sales.Customers
WHERE Score > (
    SELECT AVG(Score)
    FROM Sales.Customers
);
```

```

-- Products priced above the average product price.
SELECT *
FROM Sales.Products
WHERE Price > (
    SELECT AVG(Price)
    FROM Sales.Products
);

/*
-----
-- Correlated subquery
-- Relies on values from the main query.
-----
*/

-- Employees who earn more than the average salary in their department.
SELECT *
FROM Sales.Employees e
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Sales.Employees
    WHERE Department = e.Department
);

-- Customers who have placed at least one order.
SELECT *
FROM Sales.Customers c
WHERE EXISTS (
    SELECT 1
    FROM Sales.Orders o
    WHERE o.CustomerID = c.CustomerID
);

-- Products that have never been ordered.
SELECT *
FROM Sales.Products p
WHERE NOT EXISTS (
    SELECT 1
    FROM Sales.Orders o
    WHERE o.ProductID = p.ProductID
);

/*
-----
-- Subquery in the EXISTS clause
-----
-- Checks if a subquery returns any rows.
-- Syntax:
    SELECT column1, column2, ...
    FROM Table2
    WHERE EXISTS (
        SELECT 1
        FROM Table1
    );
*/

```

```
/* -----  
-- Example: return customers who have  
-- placed at least one order.  
----- */
```

```
SELECT *  
FROM Sales.Customers c  
WHERE EXISTS (  
    SELECT 1  
    FROM Sales.Orders o  
    WHERE o.CustomerID = c.CustomerID  
);
```

```
/* -----  
-- Example: return products that have  
-- never been ordered.  
----- */
```

```
SELECT *  
FROM Sales.Products p  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM Sales.Orders o  
    WHERE o.ProductID = p.ProductID  
);
```

```
/* -----  
-- Example: return employees who have  
-- made at least one sale.  
----- */
```

```
SELECT *  
FROM Sales.Employees e  
WHERE EXISTS (  
    SELECT 1  
    FROM Sales.Orders o  
    WHERE o.SalesPersonID = e.EmployeeID  
);
```

```
/* -----  
-- Example: return products that were  
-- ordered by any customer.  
----- */
```

```
SELECT *  
FROM Sales.Products p  
WHERE EXISTS (  
    SELECT 1  
    FROM Sales.Orders o  
    INNER JOIN Sales.Customers c  
        ON o.CustomerID = c.CustomerID  
    WHERE o.ProductID = p.ProductID  
);
```

```

/*
=====
== CTE ==
=====
- CTE stands for common table expression
- temporary, named result set (virtual table), that can be used multiple times
  within your query to simplify and organize complex query
*/

USE SalesDB;

/*
=====
TASK 1
=====
-- Show all customers from the United States using a CTE.
*/
WITH US_Customers AS
(
    SELECT *
    FROM Sales.Customers
    WHERE Country = 'USA'
)
SELECT *
FROM US_Customers;

/*
=====
TASK 2
=====
-- Show customers whose score is greater than 500.
*/
WITH High_Score_Customers AS
(
    SELECT *
    FROM Sales.Customers
    WHERE Score > 500
)
SELECT *
FROM High_Score_Customers;

/*
=====
TASK 3
=====
-- Show the CustomerID, FullName, and Country of all customers.
*/
WITH Customer_Details AS
(
    SELECT
        CustomerID,
        CONCAT_WS(' ', FirstName, LastName) AS FullName,
        Country
    FROM Sales.Customers
)

```



```
SELECT *
FROM Customer_Details;
```

```
/*
```

```
=====
TASK 4
=====
```

```
-- Find the average customer score using a CTE.
*/
```

```
WITH Average_Score AS
(
    SELECT
        AVG(Score) AS AvgScore
    FROM Sales.Customers
)
SELECT *
FROM Average_Score;
```

```
/*
```

```
=====
TASK 5
=====
```

```
-- Show customers whose score is higher than the average score.
*/
```

```
WITH Average_Score AS
(
    SELECT
        AVG(Score) AS AvgScore
    FROM Sales.Customers
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    c.Score
FROM Sales.Customers AS c
CROSS JOIN Average_Score AS a
WHERE c.Score > a.AvgScore;
```

```
/*
```

```
=====
TASK 6
=====
```

```
-- Count the number of customers in each country.
*/
```

```
WITH Country_Customers AS
(
    SELECT
        Country,
        COUNT(*) AS TotalCustomers
    FROM Sales.Customers
    GROUP BY Country
```

```
)  
SELECT *  
FROM Country_Customers  
ORDER BY TotalCustomers DESC;
```

```
/*  
=====
```

```
TASK 7  
=====
```

```
-- Show only countries that have more than 5 customers.  
*/
```

```
WITH Country_Customers AS  
(  
    SELECT  
        Country,  
        COUNT(*) AS TotalCustomers  
    FROM Sales.Customers  
    GROUP BY Country  
)  
SELECT *  
FROM Country_Customers  
WHERE TotalCustomers > 5;
```

```
/*  
=====
```

```
TASK 8  
=====
```

```
-- Find the highest customer score in each country.  
*/
```

```
WITH Country_Scores AS  
(  
    SELECT  
        Country,  
        MAX(Score) AS HighestScore  
    FROM Sales.Customers  
    GROUP BY Country  
)  
SELECT *  
FROM Country_Scores  
ORDER BY HighestScore DESC;
```

```
/*  
=====
```

```
TASK 9  
=====
```

```
-- Show customers who have the highest score in their country.  
*/
```

```
WITH Country_Scores AS  
(  
    SELECT  
        Country,  
        MAX(Score) AS HighestScore
```

```

        FROM Sales.Customers
        GROUP BY Country
    )
    SELECT
        c.CustomerID,
        c.FirstName,
        c.LastName,
        c.Country,
        c.Score
    FROM Sales.Customers AS c
    INNER JOIN Country_Scores AS cs
        ON c.Country = cs.Country
        AND c.Score = cs.HighestScore;

```

```

/*
=====

```

```

TASK 10
=====

```

```

-- Create a CTE that calculates the average score for each country.
-- Then show countries whose average score is greater than 500.
*/

```

```

WITH Country_Averages AS
(
    SELECT
        Country,
        AVG(Score) AS AverageScore
    FROM Sales.Customers
    GROUP BY Country
)
SELECT *
FROM Country_Averages
WHERE AverageScore > 500
ORDER BY AverageScore DESC;

```

```

/*

```

```

    =====
    == Views ==
    =====
    - A View is a virtual table based on the result of a SQL query, without storing
the data in database
    - views are persisted SQL queries in the database
    - table stores the data, and table is hard to maintain, that is faster response
than view
    - view stored the saved query that shows data, easy to maintain, slower response
than table

```

```

-- syntax

```

```

    CREATE VIEW view_name AS
    (
        SELECT column1, column2
        FROM table_name
        WHERE condition;
    )

```

```

-- if a table or view is created without specifying a schema, it defaults to the

```

```

DBO
*/

```

```

USE SalesDB;

-- find the total of sales of each month using aggregation functions
SELECT
    YEAR(OrderDate) AS OrderYear,
    MONTH(OrderDate) AS OrderMonth,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY
    YEAR(OrderDate),
    MONTH(OrderDate)
ORDER BY
    OrderYear ASC,
    OrderMonth ASC;

-- find the total of sales of each month using Aggregation
SELECT
    DATETRUNC(MONTH, OrderDate) AS OrderMonth,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY DATETRUNC(MONTH, OrderDate);

-- find the running total of sales of each month using CTEs
-- find the total number of orders
-- find the total number of quantities
----- these are easy with the monthly level
WITH CTE_Monthly_Summary AS (
    SELECT
        DATETRUNC(MONTH, OrderDate) AS OrderMonth,
        SUM(Sales) AS TotalSales,
        COUNT(OrderID) AS TotalOrders,
        SUM(Quantity) AS TotalQuantities
    FROM Sales.Orders
    GROUP BY DATETRUNC(MONTH, OrderDate)
)
SELECT
    OrderMonth,
    TotalSales,
    TotalOrders,
    TotalQuantities,
    SUM(TotalSales) OVER(ORDER BY OrderMonth) AS RunningTotal
FROM CTE_Monthly_Summary;

-- creating the view
CREATE VIEW Sales.V_Monthly_Summary AS
(
    SELECT
        DATETRUNC(MONTH, OrderDate) AS OrderMonth,
        SUM(Sales) AS TotalSales,
        COUNT(OrderID) AS TotalOrders,
        SUM(Quantity) AS TotalQuantities
    FROM Sales.Orders
    GROUP BY DATETRUNC(MONTH, OrderDate)
)

```

```

-- creating the view
CREATE VIEW V_Monthly_Summary AS
(
    SELECT
        DATETRUNC(MONTH, OrderDate) AS OrderMonth,
        SUM(Sales) AS TotalSales,
        COUNT(OrderID) AS TotalOrders,
        SUM(Quantity) AS TotalQuantities
    FROM Sales.Orders
    GROUP BY DATETRUNC(MONTH, OrderDate)
)

-- to find the view: open object explorer -> select the DB -> select the views
SELECT * FROM V_Monthly_Summary

-- create the view inside the specified schema
SELECT * FROM Sales.V_Monthly_Summary

-- drop existing view
DROP VIEW V_Monthly_Summary;

-- View does not exist -> CREATE the view.
-- View already exists -> ALTER (update) the view.
CREATE OR ALTER VIEW Sales.V_Monthly_Summary AS
(
    SELECT
        DATETRUNC(MONTH, OrderDate) AS OrderMonth,
        SUM(Sales) AS TotalSales,
        COUNT(OrderID) AS TotalOrders
    FROM Sales.Orders
    GROUP BY DATETRUNC(MONTH, OrderDate)
)

-- show the new data after update
SELECT *
FROM Sales.V_Monthly_Summary;

/*
    provide a view that combines details from
    orders, products, customers, and employees
*/

CREATE VIEW Sales.V_Order_Details AS
(
    SELECT
        o.OrderID,
        o.OrderDate,
        p.Product,
        p.Category,

```

```

        CONCAT_WS(' ',
            COALESCE(c.FirstName, ''),
            COALESCE(c.LastName, ''))
    ) AS CustomerName,
    c.Country AS CustomerCountry,
    CONCAT_WS(' ',
        COALESCE(e.FirstName, ''),
        COALESCE(e.LastName, ''))
    ) AS EmployeeName,
    e.Department,
    o.Sales,
    o.Quantity
FROM Sales.Orders AS o
LEFT JOIN Sales.Products AS p
    ON p.ProductID = o.ProductID
LEFT JOIN Sales.Customers AS c
    ON c.CustomerID = o.CustomerID
LEFT JOIN Sales.Employees AS e
    ON e.EmployeeID = o.SalesPersonID
);

-- access the view results
SELECT *
FROM Sales.V_Order_Details;

-----
-- example about row level security 'hide data from USA' --
-- protect few rows from the 'USA'
-- views are used to provide security to our data, whether protect our columns or rows
-----

-- provide a view for the EU sales team that combines
-- details from all tables and excludes data related to the USA
CREATE OR ALTER VIEW Sales.V_Order_Details_EU AS (
    SELECT
        o.OrderID,
        o.OrderDate,
        p.Product,
        p.Category,
        CONCAT_WS(' ',
            COALESCE(c.FirstName, ''),
            COALESCE(c.LastName, ''))
        ) AS CustomerName,
        c.Country AS CustomerCountry,
        CONCAT_WS(' ',
            COALESCE(e.FirstName, ''),
            COALESCE(e.LastName, ''))
        ) AS EmployeeName,
        e.Department,
        o.Sales,
        o.Quantity
    FROM Sales.Orders AS o
    LEFT JOIN Sales.Products AS p
        ON p.ProductID = o.ProductID
    LEFT JOIN Sales.Customers AS c
        ON c.CustomerID = o.CustomerID
    LEFT JOIN Sales.Employees AS e
        ON e.EmployeeID = o.SalesPersonID
    WHERE c.Country != 'USA'
);

-- show the view results

```

```
SELECT *  
FROM Sales.V_Order_Details_EU  
ORDER BY CustomerCountry;
```

```
/*  
    Views can be used as data marts in data warehouse system, because they  
    provide a flexible and efficient way to present data  
*/
```

```
/*  
=====
```

== Advantages ==

```
=====
```

- Reusable:
Write once and use many times.
- Performance:
SQL Server can reuse cached execution plans, which may improve performance in some cases.
- Secure:
Control who can execute the procedure by managing permissions.
- Parameters:
Can accept different input values, allowing the same procedure to work with different data.
- Easy to maintain:
Keep related SQL logic in one place.
- Useful for ETL:
Good for data loading, transformations, validation, logging, incremental loading, and error handling.

```
*/
```

```
/*  
=====
```

== Disadvantages ==

```
=====
```

- Can be hard to maintain:
Large or poorly designed procedures can become complicated.
- Database dependent:
Stored Procedure syntax and features can vary between different database management systems.
- Harder to test:
Complex procedures can be difficult to test.
- Harder to debug:
Debugging complex database logic can sometimes be more difficult than debugging application code.
- Can be overused:
Don't put all application logic inside Stored Procedures.

```
*/
```

```
USE SalesDB;
GO
```

```
/*
```

```
=====
== Stored Procedure ==
=====
```

- A Stored Procedure is a saved collection of SQL statements stored inside the database.

- You can execute the Stored Procedure whenever you need it.

- Think of it as a function-like object in SQL:

- input -> parameters
 - logic -> SQL statements
 - output -> result / modified data

- Stored Procedures can contain:

- SELECT
 - INSERT
 - UPDATE
 - DELETE
 - IF / ELSE
 - WHILE
 - Variables
 - Temporary Tables
 - Transactions
 - Error Handling
 - Other Stored Procedures

- In Data Warehouse / ETL projects, Stored Procedures are commonly used to organize and execute ETL steps such as:

- * Loading data from Bronze to Silver
 - * Cleaning and transforming data
 - * Loading data from Silver to Gold
 - * Incremental loading
 - * Data validation
 - * Logging ETL operations
 - * Error handling

- Keep Stored Procedures focused on a specific database operation or ETL task instead of creating one huge procedure.

```
*/
```

```
-----
-- We locate our stored procedures:
-- Object Explorer -> Database -> Programmability -> Stored Procedures
-----
```

```
-----
-- For USA customers, find the total number of customers and the average score
-----
```

```
-- STEP 1: Write and test the query
```



```
SELECT
    COUNT(*) AS TotalCustomers,
    AVG(Score) AS AverageScore
FROM Sales.Customers
WHERE Country = 'USA';
```

-- STEP 2: Turn the query into a Stored Procedure

```
CREATE OR ALTER PROCEDURE GetCustomerSummary
AS
BEGIN
```

```
    SELECT
        COUNT(*) AS TotalCustomers,
        AVG(Score) AS AverageScore
    FROM Sales.Customers
    WHERE Country = 'USA';
```

```
END;
GO
```

-- STEP 3: Execute the Stored Procedure

```
EXEC GetCustomerSummary;
GO
```

-- Stored Procedure With Parameters --

-- Parameter:
-- A parameter is a placeholder used to pass a value
-- from the caller to the Stored Procedure.
--
-- Parameters allow the same Stored Procedure to process
-- different input values dynamically.

-- For customers from a specific country, find the total number of customers
-- and the average score

-- STEP 1: Alter the existing Stored Procedure and add a parameter

```
CREATE OR ALTER PROCEDURE GetCustomerSummary
    @Country NVARCHAR(50)
AS
BEGIN
```

```
    SELECT
        COUNT(*) AS TotalCustomers,
        AVG(Score) AS AverageScore
    FROM Sales.Customers
    WHERE Country = @Country;
```

```
END;
GO
```

-- STEP 2: Execute the Stored Procedure with different parameters

```
EXEC GetCustomerSummary @Country = 'Germany';  
GO
```

```
EXEC GetCustomerSummary @Country = 'Egypt';  
GO
```

-- Drop Procedure --

```
DROP PROCEDURE GetCustomerSummary;  
GO
```

-- Default Value --

-- A default parameter automatically uses a predefined value
-- when the caller does not provide a value.
--
-- This makes the parameter optional.

```
CREATE OR ALTER PROCEDURE GetCustomerSummary  
    @Country NVARCHAR(50) = 'USA'  
AS  
BEGIN
```

```
    SELECT  
        COUNT(*) AS TotalCustomers,  
        AVG(Score) AS AverageScore  
    FROM Sales.Customers  
    WHERE Country = @Country;
```

```
END;  
GO
```

-- Execute without providing @Country.
-- The default value 'USA' will be used.

```
EXEC GetCustomerSummary;  
GO
```

-- Provide a different value.
-- The provided value 'Egypt' overrides the default value.

```
EXEC GetCustomerSummary @Country = 'Egypt';  
GO
```

-- Stored Procedure - Multiple Statements --

-- A Stored Procedure can contain multiple SQL statements
-- that are executed together when the procedure is called.
-- Must add semicolon at the end of each SQL statement
-- A Stored Procedure can also return multiple result sets.

```

CREATE OR ALTER PROCEDURE GetCustomerSales
    @CustomerID INT
AS
BEGIN

    -- Statement 1: Get customer information

    SELECT *
    FROM Sales.Customers
    WHERE CustomerID = @CustomerID;

    -- Statement 2: Get customer's orders

    SELECT *
    FROM Sales.Orders
    WHERE CustomerID = @CustomerID;

    -- Statement 3: Calculate total number of orders
    -- and total sales

    SELECT
        @CustomerID AS CustomerID,
        COUNT(*) AS TotalOrders,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    WHERE CustomerID = @CustomerID;

END;
GO

-- Execute the Stored Procedure

EXEC GetCustomerSales @CustomerID = 100;
GO

-----
-- Stored Procedure - Variables --
-----
-- placeholders used to store values to be used later in the procedure
-- 'parameters' pass values into a stored procedure or return values back to the caller
-- 'variables' temporarily store and manipulate data during its execution

-- total customers from Germany = 2
-- average score from Germany = 425

CREATE OR ALTER PROCEDURE GetCustomerSummary
    @Country NVARCHAR(50) = 'USA'
AS
BEGIN

    -- Declare variables
    DECLARE @TotalCustomers INT,
            @AverageScore FLOAT;

    -- Assign values to variables
    SELECT

```

```

        @TotalCustomers = COUNT(*),
        @AverageScore = AVG(Score)
FROM Sales.Customers
WHERE Country = @Country;

-- Display customer summary
PRINT 'Total Customers from '
      + @Country
      + ': '
      + CAST(@TotalCustomers AS NVARCHAR);

PRINT 'Average Score from '
      + @Country
      + ': '
      + CAST(@AverageScore AS NVARCHAR);

-- Calculate total orders and total sales
SELECT
    COUNT(o.OrderID) AS TotalOrders,
    SUM(o.Sales) AS TotalSales
FROM Sales.Orders AS o
INNER JOIN Sales.Customers AS c
    ON o.CustomerID = c.CustomerID
WHERE c.Country = @Country;

END;
GO

-- Execute using the default value: USA
EXEC GetCustomerSummary;
GO

-- Execute using Germany
EXEC GetCustomerSummary @Country = 'Germany';
GO

```

```

/*
=====
== Trigger ==
=====
- A trigger is a special type of stored procedure that automatically executes when a
specific event happens on a table or view.
- Trigger = automatic SQL code that runs when a specific database event occurs
- A common use case is auditing/logging—for example, automatically recording
who/what was inserted, updated, or deleted.
- common trigger events
    INSERT
    UPDATE
    DELETE
- SQL Server provides special virtual tables inside triggers
    INSERTED: contains the new rows
    DELETED: contains the old/deleted rows
*/

USE SalesDB;
GO

```

```
-- Create log table
CREATE TABLE Sales.EmployeeLogs (
    LogID INT IDENTITY(1, 1) PRIMARY KEY,
    EmployeeID INT,
    LogMessage VARCHAR(100),
    LogDate DATE
);
GO

-- Create the trigger on Employees table
CREATE TRIGGER trg_afterInsertEmployee
ON Sales.Employees
AFTER INSERT
AS
BEGIN
    INSERT INTO Sales.EmployeeLogs (EmployeeID, LogMessage, LogDate)
    SELECT
        EmployeeID,
        'New Employee Added = ' + CAST(EmployeeID AS NVARCHAR),
        GETDATE()
    FROM INSERTED;
END;
GO

-- insert new records
INSERT INTO Sales.EmployeeLogs
(EmployeeID, LogMessage, LogDate)
VALUES
(41, 'New Employee Added = 41', GETDATE()),
(42, 'New Employee Added = 42', GETDATE()),
(43, 'New Employee Added = 43', GETDATE()),
(44, 'New Employee Added = 44', GETDATE());

-- show the result
SELECT *
FROM Sales.EmployeeLogs ;
GO

-- delete trigger
DROP TRIGGER IF EXISTS trg_afterInsertEmployee;
GO
```
